

GOLDEN HOUSE

Architecture Microservices & Spécifications Techniques

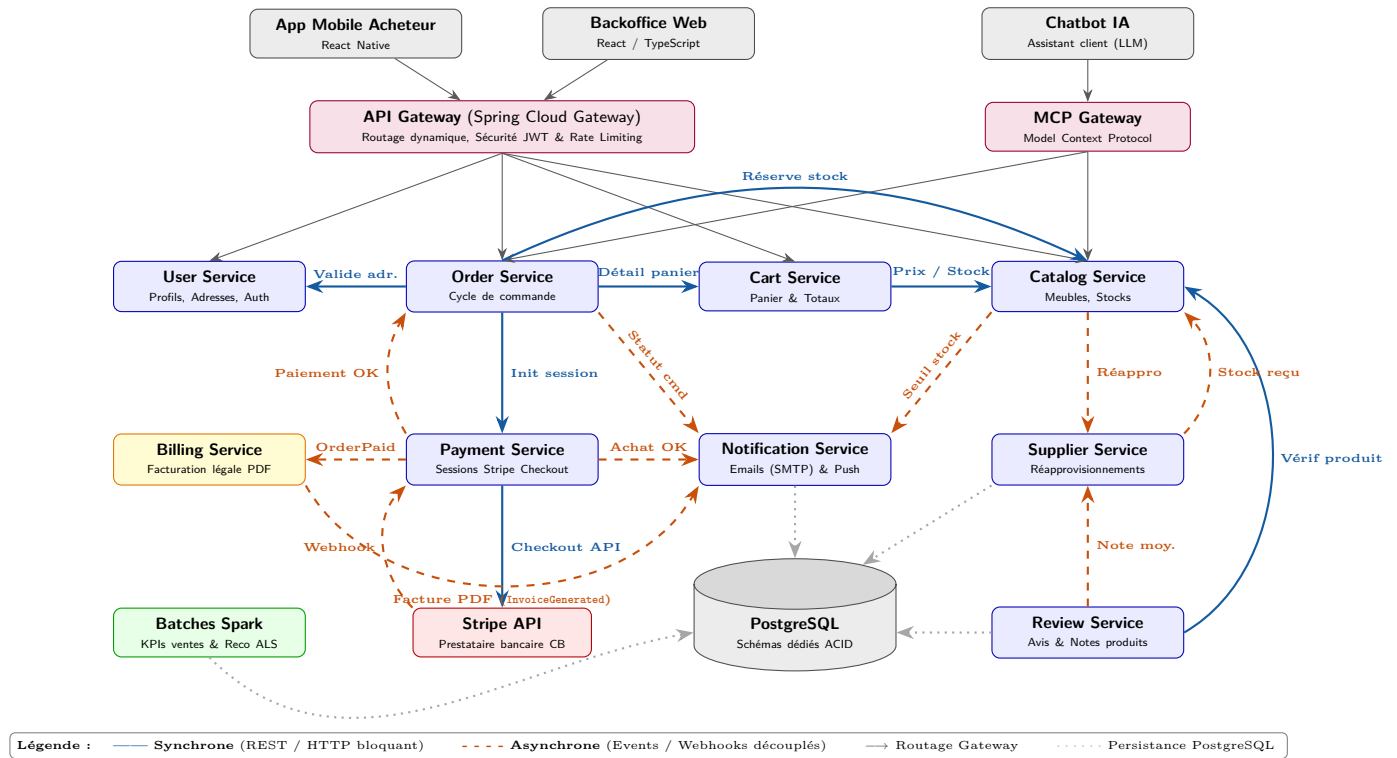
Auteurs : Justin Chapon, Nabil Bennacer, Tom Cantillon, Salma El Annaby, Manuela Zapata Quirós

Polytech Montpellier – IG5 / DaMS

Module : **IWA 2** (Christophe Nauroy)

Semestre 9 – Septembre 2026

1. Schéma d'Architecture Globale des Microservices



2. Typologie des Échanges Inter-Services

- **Communication Synchrone (REST / HTTP)** : Requêtes bloquantes exigeant une réponse immédiate pour finaliser l'action courante (lecture temps réel du catalogue, calcul et figeage du panier, validation de l'adresse acheteur, réservation atomique des stocks, création de session Stripe).
- **Communication Asynchrone (Events / Webhooks)** : Traitements découplés dans le temps et tolérants aux pannes sans bloquer l'acheteur (réception webhook Stripe, confirmation de paiement vers commande, génération différée de la facture légale PDF, notifications email/push, réassort fournisseur, recalcul de note).

3. Matrice Exhaustive des Flux Inter-Services

Source	Cible	Type	Protocole / Signal	Justification Technique & Métier
Clients (Mobile/Web)	API Gateway	Sync	HTTP/2 / REST	Point d'entrée unique, terminaison TLS, validation JWT.
API Gateway	Microservices	Sync	HTTP REST interne	Routage inverse vers les services cibles selon l'URL.
Chatbot IA	MCP Gateway	Sync	JSON-RPC / REST	Traduction du langage naturel en requêtes microservices.
Order	User	Sync	GET /api/users/{id}	Contrôle bloquant de l'existence client et adresse de livraison.
Order	Cart	Sync	GET /api/cart/{id}	Récupération synchrone du snapshot panier (items, totaux) à figer.
Cart	Catalog	Sync	GET /api/products/{id}	Vérification en temps réel du stock disponible et du prix unitaire.
Order	Catalog	Sync	POST /products/reserve	Verrouillage atomique des stocks avant confirmation du devis.
Order	Payment	Sync	POST /payments/init	Création de la session d'encaissement et obtention URL Stripe.
Payment	Stripe	Sync	HTTPS / REST	Appel synchrone à l'API Stripe pour initialiser le Checkout.
Review	Catalog	Sync	GET /api/products/{id}	Validation d'existence du produit avant enregistrement de l'avis.
Stripe	Payment	Async	Webhook checkout.session	Signal bancaire externe reçu hors session utilisateur.
Payment	Order	Async	Event PaymentConfirmed	Notification de règlement : transition de commande vers PAID.
Payment	Billing	Async	Event OrderPaidEvent	Déclenchement autonome de la génération de facture (non bloquant).
Billing	Notification	Async	Event InvoiceGenerated	Transmission de l'URL du PDF de facture pour envoi à l'acheteur.
Order	Notification	Async	Event OrderStatusChanged	Notification client des étapes de traitement (expédiée, livrée).
Payment	Notification	Async	Event PaymentSucceeded	Envoi immédiat de l'email de confirmation d'achat avec le reçu.
Catalog	Notification	Async	Event StockBelowThreshold	Alerte automatique du gestionnaire en cas de stock critique.
Catalog	Supplier	Async	Event RestockNeededEvent	Émission automatique d'une demande de réapprovisionnement.
Supplier	Catalog	Async	Event StockReceivedEvent	Incrémentation asynchrone des stocks à réception fournisseur.
Review	Supplier	Async	Event RatingAggregated	Recalcul asynchrone de la note de qualité moyenne fournisseur.
Batches	Order / Catalog	Async	Jobs Spark SQL	Traitements nocturnes pour agrégats KPIs et recommandations ALS.

4. Points d'Entrée & Modèles de Données – Services 1 à 5

4.1. API Gateway & MCP Gateway (Passerelles d'accès)

- **Points d'Entrée :**
 - ANY /api/v1/** : Routage dynamique vers microservices, terminaison TLS, validation token JWT.
 - POST /mcp/v1/query : Requête contextuelle Chatbot IA (Model Context Protocol).
- **Modèle de Données :** *Stateless*. Cache Redis pour la révocation des tokens JWT et le rate limiting.

4.2. User Service (Utilisateurs, Adresses & Authentification)

- **Points d'Entrée :**
 - POST /api/users/register : Inscription nouvel utilisateur.
 - POST /api/users/login : Authentification et génération du token JWT.
 - GET | PUT /api/users/{id} : Consultation et mise à jour du profil utilisateur.
 - GET | POST /api/users/{id}/addresses : Gestion des adresses de facturation et de livraison.
- **Modèle de Données (PostgreSQL) :**
 - **users :** id (UUID, PK), email (VARCHAR, UK), username (VARCHAR, UK), password_hash, first_name, last_name, phone, role (role_enum), currency (currency_enum), stripe_customer_id (UK), is_active (BOOL).
 - **addresses :** id (UUID, PK), user_id (UUID, FK users), type (address_type), street, postal_code, city, country, is_default (BOOL).

4.3. Catalog Service (Produits, Mobilier & Gestion des Stocks)

- **Points d'Entrée :**
 - GET /api/products : Liste paginée des meubles avec filtres (catégorie, matière, couleur, prix).
 - GET /api/products/{id} : Fiche détaillée d'un meuble (dimensions, stock disponible).
 - POST /api/products : Création / mise à jour produit (réservé Backoffice).
 - POST /api/products/reserve : Réservation atomique de quantité lors de la commande.
- **Modèle de Données (PostgreSQL) :**
 - **products :** id (UUID, PK), nom (VARCHAR), prix (NUMERIC), categorie (category_enum), type_matiere (material_enum), couleur, largeur/hauteur/prof. (INT), stock (INT), seuil_alerte (INT), images (TEXT[]), fournisseur_id (UUID, FK), est_actif (BOOL).

4.4. Cart Service (Gestion des Paniers d'Achat)

- **Points d'Entrée :**
 - GET /api/cart/{userId} : Récupération du panier actif et calcul des sous-totaux.
 - POST /api/cart/items : Ajout d'un article dans le panier.
 - PUT /api/cart/items/{id} : Modification de quantité d'un article.
 - DELETE /api/cart/items/{id} : Suppression d'un article; DELETE /api/cart/{id} : Vidage du panier.
- **Modèle de Données (PostgreSQL) :**
 - **carts :** id (UUID, PK), user_id (UUID, FK-UQ users), code_promo (VARCHAR).
 - **cart_items :** id (UUID, PK), cart_id (UUID, FK carts), product_id (UUID, FK products), quantite (INT).

4.5. Order Service (Commandes & Cycle de Vie)

- **Points d'Entrée :**
 - POST /api/orders : Création de commande depuis le panier, validation adresse et réservation stock.
 - GET /api/orders/{id} : Détail d'une commande (lignes, montants, adresses, statuts).
 - GET /api/orders/user/{userId} : Historique complet des commandes de l'acheteur.
 - PUT /api/orders/{id}/status : Mise à jour du statut (PENDING, PAID, SHIPPED, DELIVERED).
- **Modèle de Données (PostgreSQL) :**
 - **orders :** id (UUID, PK), order_number (VARCHAR, UK), user_id (UUID, FK), statut (order_status_enum), statut_paiement (payment_enum), stripe_payment_intent_id (VARCHAR, UK), shipping_address_id (UUID, FK), billing_address_id (UUID, FK), montant_total (NUMERIC), devise (currency_enum).
 - **order_items :** id (UUID, PK), order_id (UUID, FK orders), product_id (UUID, FK products), nom_produit (VARCHAR), quantite (INT), prix_unitaire (NUMERIC), total_ligne (NUMERIC).

5. Points d'Entrée & Modèles de Données – Services 6 à 10

5.1. Payment Service (Passerelle Stripe & Encaissements)

- **Points d'Entrée :**
 - POST /api/payments/init : Création de la session Stripe Checkout et renvoi de l'URL de paiement.
 - POST /api/payments/webhook : Réception asynchrone des événements de paiement Stripe.
 - GET /api/payments/order/{orderId} : Consultation de l'état de la transaction financière.
- **Modèle de Données (PostgreSQL) :**
 - **payments** : id (UUID, PK), order_id (UUID, UK), stripe_session_id, stripe_payment_intent_id, amount (NUMERIC), currency, status (payment_enum), paid_at (TIMESTAMPTZ).

5.2. Billing Service (Facturation & Génération Fiscale)

- **Composant Métier :** Consommateur asynchrone de l'événement OrderPaidEvent. Attribue un numéro fiscal chronologique unique, calcule la ventilation HT / TVA / TTC, et compile le document légal PDF stocké sur S3 / MinIO.
- **Points d'Entrée :**
 - GET /api/invoices/order/{orderId} : Récupération des métadonnées et URL du PDF de facture.
 - GET /api/invoices/{id}/pdf : Téléchargement direct du document PDF certifié.
 - GET /api/invoices/user/{userId} : Liste des factures de l'acheteur pour consultation comptable.
- **Modèle de Données (PostgreSQL) :**
 - **invoices** : id (UUID, PK), numero_facture (VARCHAR, UK, ex : FACT-2026-0001), order_id (UUID, FK-UQ orders), user_id (UUID, FK users), montant_ht (NUMERIC), taux_tva (NUMERIC), montant_tva (NUMERIC), montant_ttc (NUMERIC), url_pdf (VARCHAR), date_facturation (TIMESTAMPTZ).

5.3. Notification Service (Emails SMTP & Push Mobiles)

- **Points d'Entrée :**
 - POST /api/notifications/send : Envoi direct déclenché par les services internes.
 - GET /api/notifications/user/{userId} : Centre de notifications in-app pour l'application mobile.
 - PUT /api/notifications/{id}/read : Marquage de lecture d'une notification.
- **Modèle de Données (PostgreSQL) :**
 - **notifications** : id (UUID, PK), user_id (UUID, FK users), titre (VARCHAR), type (notif_type_enum : EMAIL, PUSH), est_lu (BOOL).

5.4. Supplier Service (Fournisseurs & Réapprovisionnements)

- **Points d'Entrée :**
 - GET /api/suppliers : Annuaire des fabricants partenaires et coordonnées.
 - POST /api/suppliers/orders : Émission d'un ordre d'achat automatique ou manuel suite à alerte stock.
 - PUT /api/suppliers/orders/{id}/receive : Réception de livraison et déclenchement d'incrément de stock.
- **Modèle de Données (PostgreSQL) :**
 - **fournisseurs** : id (UUID, PK), nom (VARCHAR), email (VARCHAR), telephone (VARCHAR), adresse (TEXT).
 - **supply_orders** : id (UUID, PK), fournisseur_id (UUID, FK), statut (supply_status_enum), montant_total (NUMERIC), date_emission (TIMESTAMPTZ), date_reception (TIMESTAMPTZ).
 - **supply_order_items** : id (UUID, PK), supply_order_id (UUID, FK), product_id (UUID, FK), quantite (INT), prix_unitaire (NUMERIC).

5.5. Review Service & Batches Spark (Avis Clients & Analyse Données)

- **Review Service (Avis Clients) :**
 - **Points d'Entrée :** POST /api/reviews (dépôt avis vérifié), GET /api/reviews/product/{id} (avis du produit).
 - **Modèle PostgreSQL (reviews) :** id (UUID, PK), user_id (UUID, FK), product_id (UUID, FK), titre (VARCHAR), note (INT 1-5), reponse_commercant (TEXT).
- **Batches Spark (Moteur Analytique & Recommandation) :**
 - **Traitements :** Traitements Spark SQL distribués de nuit (chiffre d'affaires, panier moyen, volumes par catégorie) et entraînement du modèle de recommandation collaborative ALS (Alternating Least Squares).
 - **Modèle PostgreSQL :**
 - **product_recommendations** : user_id, product_id, score (FLOAT).
 - **kpis_ventes_journalieres** : date, nb_commandes, chiffre_affaires_ht.